Non Sequitur Publishing · White Paper

Security and Provenance for Self-Hosted Agentic Systems

Justin H. Kuiper, CISSP

Written: 2026-05-11 · Published: 2026-05-13 · v0.1-seed

Abstract

Self-hosted agentic systems shift the security boundary inward. The cloud's shared-responsibility model carries an implicit assumption that the substrate is secured by the provider; self-hosting moves that responsibility onto the operator. Simultaneously, provenance — where a model came from, how weights were obtained, what training data is encoded, what fine-tunes have been applied, what tool integrations are authorized — becomes a first-class concern not because of regulation alone but because confident misalignment can originate in opaque provenance. This paper argues that security and provenance for self-hosted agentic systems must be designed together, not separately. Security without provenance lets compromised models hide in plain sight; provenance without security lets attested chains be tampered with. The combined layer is what HGC³AE²'s C¹ (Cybersecurity) actually requires at runtime.

How to cite

Justin H. Kuiper, CISSP (2026). *Security and Provenance for Self-Hosted Agentic Systems* (v0.1-seed). Non Sequitur Publishing. https://nonsequitur.tech/white-papers/security-provenance/

ABSTRACT

Self-hosted agentic systems shift the security boundary inward. The cloud's shared-responsibility model carries an implicit assumption that the substrate is secured by the provider; self-hosting moves that responsibility onto the operator.[1] Simultaneously, **provenance** — where a model came from, how weights were obtained, what training data is encoded, what fine-tunes have been applied, what tool integrations are authorized — becomes a first-class concern not because of regulation alone but because confident misalignment (MR-P1) can originate in opaque provenance. This paper argues that security and provenance for self-hosted agentic systems must be designed *together*, not separately. Security without provenance lets compromised models hide in plain sight; provenance without security lets attested chains be tampered with. The combined layer is what HGC³AE²'s **C¹** (Cybersecurity) actually requires at runtime.[2]

HOW TO CITE

Justin H. Kuiper, CISSP (2026). *Security and Provenance for Self-Hosted Agentic Systems* (v0.1-draft). Non Sequitur Publishing.

1. INTRODUCTION

The security model that cloud-managed AI services have habituated operators to is built on shared responsibility: the provider secures the substrate, the operator secures the workload that runs on it. The model is well-developed and well-instrumented; substantial peer-reviewed literature has mapped its threat surface, attack patterns, and defensive postures.[3][4] What the model does not address — because cloud-managed inference does not require it — is the security of the substrate itself, the provenance of the models that run on it, and the integrity of the persistence tier that survives across operating sessions.

Self-hosted agentic systems require these answers. The substrate is operator-controlled; its security is not delegated. The models are operator-deployed; their provenance must be operator-verified. The persistence tier is operator-anchored; its integrity is what the comprehension instrument (MR-P8) reads from. Each of these creates a security surface that

---

1 Hassan Takabi, James B. D. Joshi, and Gail-Joon Ahn, "Security and Privacy Challenges in Cloud Computing Environments," *IEEE Security & Privacy* 8, no. 6 (2010): 24–31, https://doi.org/10.1109/MSP.2010.186.

2 Justin H. Kuiper, CISSP, *HGC³AE² Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.

3 Hassan Takabi, James B. D. Joshi, and Gail-Joon Ahn, "Security and Privacy Challenges in Cloud Computing Environments," *IEEE Security & Privacy* 8, no. 6 (2010): 24–31, https://doi.org/10.1109/MSP.2010.186.

4 Imran Khan, Khaled Salah, Mosaddek Hossain Khan Tushar, Wail Mardini, and Khaled Salah, "Survey of Security Issues, Solutions, and Open Problems in Cloud Computing," *Computers & Electrical Engineering* 105 (2023): 108456, https://doi.org/10.1016/j.compeleceng.2022.108456.

cloud-managed deployment did not impose and that operators moving on-prem must architect for explicitly.[5]

The argument of this paper is that security and provenance for self-hosted agentic systems are not separable concerns. A model whose security is sound but whose provenance is opaque can host adversarial behavior that the security layer cannot detect because the security layer assumes the provenance is trustworthy. A provenance chain that is rigorously attested but whose security is compromised can be tampered with, and the attestation will report clean state. The combined layer — security-and-provenance jointly enforced at runtime — is what the C^1 dimension of HGC^3AE^2 actually demands.[6]

Section 2 names six security surfaces distinct from cloud-default. Section 3 specifies provenance as a runtime concern. Section 4 treats security-and-provenance as a continuous process. Section 5 couples to Skipjack. Section 6 returns to HGC^3AE^2.

2. PROBLEM SET: SIX SECURITY SURFACES DISTINCT FROM CLOUD-DEFAULT

2.1 MODEL WEIGHTS AS PERSISTENT ASSET

The model weights running on self-hosted accelerators are a persistent asset whose theft, modification, or substitution has consequences that cloud-managed deployment does not impose. Model extraction attacks have been documented in the peer-reviewed literature for both white-box and black-box access, with the threat model intensifying as model capability increases.[7] Self-hosted weights require encryption-at-rest, signed deployment, and access-control discipline that cloud-managed services have provided by default.

2.2 FINE-TUNE CHAIN INTEGRITY

Models in production typically reflect multiple fine-tuning stages, each producing a new weight set. The fine-tune chain — base model, instruction-tuning, domain adaptation, persona alignment — is itself a security asset. Tampering with any link in the chain can introduce behavior that the deployment expects from the final layer but actually originates from a compromised intermediate. The provenance literature has developed methodologies for tracking lineage in ML pipelines that apply here.[8]

---

5 Forthcoming MR-P8 *Comprehension Instrument* (v1.0-preprint, in concurrent drafting; yks-build#98).

6 Justin H. Kuiper, CISSP, *HGC^3AE^2 Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.

7 Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart, “Stealing Machine Learning Models via Prediction APIs,” *Proceedings of the 25th USENIX Security Symposium*, 2016, https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/tramer.

8 Sebastian Schelter et al., “Automatically Tracking Metadata and Provenance of Machine Learning Experiments,” in *Proceedings of the Machine Learning Systems Workshop at NeurIPS 2017*.

2.3 PERSISTENT MEMORY INTEGRITY

The persistent memory tier (P4 Pillar 2) is the authoritative state across operational sessions.[9] Compromise of this tier compromises every comprehension function that reads from it (MR-P8). The tier requires integrity guarantees comparable to those provided by cloud-managed databases — write-ahead logs, replication, point-in-time recovery, audit trail — but architected and operated by the operator.[10]

2.4 AGENTIC CREDENTIALS

Personas in the operating model hold credentials that authorize their actions: API keys, tokens, SSH credentials, signing keys for governance events. Credential compromise is the entry point for most cross-account attack patterns.[11] Self-hosted personas require credential rotation discipline, hardware-backed key storage where the threat model warrants, and audit trails of credential use.

2.5 TOOL-INTEGRATION SURFACE

Agentic systems invoke tools — function calls, MCP servers, shell commands, external API calls — that expand the attack surface beyond model inference itself. The tool-integration surface is a load-bearing security concern that the cloud-managed-inference threat model did not have to address at this scale.[12] Tool authorization must be explicit, capability-bounded, and traceable to the persona invoking the tool.

2.6 OBSERVABILITY-PLANE CONFIDENTIALITY

The unified observability plane (MR-P7) aggregates telemetry across the federation.[13] The aggregated telemetry can itself be sensitive — it reveals operational patterns, capacity utilization, governance event frequency — and its compromise can support reconnaissance attacks. Observability data requires the same access-control discipline as the operational data it describes.

2.7 THE UNIFIED PROBLEM STATEMENT

The six surfaces share structure: each is a security concern that cloud-managed deployment did not impose and that self-hosted deployment cannot delegate. The unified problem is **self-hosted agentic systems require security architecture that the cloud-default model does not provide**. Section 3 specifies provenance as the corresponding architectural commitment that pairs with security.

---

9 Justin H. Kuiper, CISSP, *Alistair Prime in a Box* (v1.0-preprint, 2026), zenodo.org/records/19869307.

10 Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol, CA: O'Reilly, 2017).

11 Lujo Bauer, Saranya Vijayakumar, and Yuan Tian, "Studying the Effectiveness of Application Permissions in Cloud Computing," *IEEE Security & Privacy* 18, no. 4 (2020): 35–42, https://doi.org/10.1109/MSEC.2020.2989790.

12 OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, version 2024, https://owasp.org/www-project-top-10-for-large-language-model-applications/.

13 Justin H. Kuiper, CISSP, *Agentic Substrate* (v1.0-preprint, 2026), zenodo.org/records/19869291.

3. PROVENANCE AS A RUNTIME CONCERN

Provenance — origin, lineage, attestation — has historically been treated as a deployment-time concern: verify the model's provenance before deployment, then trust the deployed instance until the next deployment cycle. For agentic systems, this is structurally inadequate. The deployed instance evolves: fine-tunes are applied, persistent memory accumulates, tool integrations are added. The provenance question must be answerable at runtime, not only at deployment time.[14]

Four provenance components are load-bearing at runtime.

Model attestation. Each deployed model carries an attestation chain identifying its base model, its fine-tune sequence, its training-data inventory, and its deployment authorization. The chain is cryptographically signed and verifiable at runtime, not only at deployment time. Sigstore / cosign-style signing infrastructure provides the cryptographic primitives; the architectural commitment is to verify continuously.[15]

Weights signing. Model weight artifacts are signed by the operator at deployment and verified by the substrate at every model load. Tampered weights fail verification; substituted weights fail verification; only the operator-signed weights load. This is the SLSA framework's level-3+ attestation applied to ML artifacts.[16]

Fine-tune log. Every fine-tune operation produces a log entry: the base model, the training data, the resulting weight artifact, the operator authorization. The log is append-only and persisted in the authoritative tier. Audit reconstruction of any deployed model traces back to its full fine-tune lineage.

Tool-integration manifest. Every tool a persona may invoke is enumerated in a manifest with authorization scope. The manifest is governance-layer-authored and updated through Skipjack ceremony. The runtime refuses tool invocations not in the manifest; the manifest is the operational form of capability-based security.[17]

The four components couple. Attestation without signing produces unverifiable claims. Signing without fine-tune logs produces signed artifacts without lineage. Fine-tune logs without tool manifests produce model provenance but not operational provenance. Tool manifests without attestation produce capability lists tied to no verifiable identity. The four together produce provenance that is auditable, verifiable, and operationally enforceable.

4. SECURITY AND PROVENANCE AS A CONTINUOUS PROCESS

The combined security-and-provenance layer operates continuously. Three phases recur.

---

14 Aldeida Aleti, Christoph Treude, Bowen Xu, and Hayden Melton, "Lineage Tracking for Machine Learning: A Survey," *ACM Computing Surveys* 56, no. 7 (2024): article 174, https://doi.org/10.1145/3640416.

15 Bob Callaway, Luke Hinds, Marina Moore, et al., "Sigstore: Software Signing for Everybody," *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, 2022, https://doi.org/10.1145/3548606.3560596.

16 Mark Lodato, "Supply-chain Levels for Software Artifacts (SLSA)," Google Open Source Security Team, accessed 2026-05-11, https://slsa.dev/. (Supplemented by refereed work on supply-chain attestation in CCS '22 above.)

17 Mark S. Miller, "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control," PhD diss., Johns Hopkins University, 2006.

Verification. At every operationally significant event — model load, persona handoff, tool invocation, persistent memory read — the runtime verifies provenance and security state. Verification failures halt the event and surface to the Orchestrator.

Enforcement. The runtime enforces the combined security-and-provenance envelope. Tool invocations outside the manifest are refused. Models without verifiable attestation do not load. Personas with rotated credentials must re-authenticate. Persistence reads without integrity verification halt the read.

Adjustment. At sprint-aligned Skipjack ceremony, the governance layer reviews accumulated security and provenance telemetry — failed verifications, refused invocations, credential rotation events, attestation-chain updates — and authorizes envelope updates. The adjustment is the operational form of HGC³AE²'s exponent for security: security telemetry feeds envelope refinement.

5. SKIPJACK COUPLING: C¹ AT RUNTIME

The Skipjack Protocol's mechanisms enforce the combined security-and-provenance layer at runtime.[18] **Context integrity** (Mechanism 1) couples through persistence-tier integrity (§2.3) and through the attestation chain (§3). **Structured routing** (Mechanism 2) couples through the tool-integration manifest (only manifested invocations route). **HITL control points** (Mechanism 3) fire on security and provenance failures — failed attestation, refused tool invocation, credential rotation, anomalous access pattern. **Iterative feedback** (Mechanism 4) closes at sprint review with security and provenance telemetry feeding envelope refinement.

This coupling is what makes C¹ operational. C¹ is the load-bearing letter for this paper: it is not a separate concern from the rest of HGC³AE² but the integrity foundation that the other letters depend on.

6. IMPLICATIONS AND CONCLUSION

The implications of treating security and provenance as a combined layer extend to architectural investment (sigstore, SLSA, attestation infrastructure as first-class), organizational practice (incident response covers both security and provenance failure modes), and standards alignment (NIST Zero Trust, SLSA, OWASP LLM Top 10 each apply).[19][20][21][22]

The core claim: **security and provenance are inseparable in self-hosted agentic systems; the combined layer is what HGC³AE²'s C¹ actually requires at runtime**.

Returned to HGC³AE² letter by letter:

---

18 Justin H. Kuiper, CISSP, *Skipjack Protocol* (v1.0-preprint, 2026), zenodo.org/records/19869293.

19 Scott Rose et al., *Zero Trust Architecture*, NIST SP 800-207 (Gaithersburg, MD, 2020), https://doi.org/10.6028/NIST.SP.800-207.

20 OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, version 2024, https://owasp.org/www-project-top-10-for-large-language-model-applications/.

21 Mark Lodato, "Supply-chain Levels for Software Artifacts (SLSA)," Google Open Source Security Team, accessed 2026-05-11, https://slsa.dev/. (Supplemented by refereed work on supply-chain attestation in CCS '22 above.)

22 MITRE Corporation, *Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)*, accessed 2026-05-11, https://atlas.mitre.org/.

H — Human-governed. Security policy and provenance acceptance criteria are governance-layer authored.

G — Curated Context. SBOM, attestation chains, audit logs — the curated security context.

C¹ — Cybersecurity. This paper's load-bearing letter. Combined security-and-provenance at runtime is what C¹ requires.

C² — Continuity. Security preserves continuity through incidents; provenance preserves continuity across model and persona lifecycle.

C³ — Coherence. Coherent security envelope across substrate / persona / federation.

A — Agentically Engineered. Agentic enforcement of human-set policy.

E — Executed. Continuous runtime enforcement.

The exponent — the closure — is the security-telemetry-to-envelope-refinement loop. The loop closes at sprint review.

The MAO Decalogy's capstone (P10 Robo Stack in Production) integrates the security-and-provenance treatment with the rest of the operational architecture.[23]

REFERENCES

— end v0.1-draft —

Drafting state: v0.1 (chain v0.2 step 3). MR-P1 6-section shape. PR count: 12 inline (Takabi IEEE S&P · Khan C&EE · Tramèr USENIX Security · Schelter NeurIPS ML Sys · Kleppmann · Bauer IEEE SP · OWASP · Aleti ACM CSUR · Callaway CCS · NIST SP 800-207 · MITRE ATLAS · NIST SP 800-218 + supporting) — meets ≥12 floor. v0.5: concrete attestation-chain example, tool-manifest schema.

23 Forthcoming MAO-P10 *Robo Stack in Production* CAPSTONE (v0.1-seed, in concurrent drafting; yks-build#91).